

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.2

FIELD TYPES

 Copy as Markdown

Field types

Every model field has a YAML type that becomes a Dart type, a JSON encoding, and, on a [table](#) model, a database column. This catalog lists the mapping, allowed defaults, query operators, and storage caveats for each type. For the model file format, see [Working with models](#).

Overview

YAM L	Dart	Postgres	SQLite	JSON	Defaults
<code>bool</code>	<code>bool</code>	<code>boolean</code>	<code>INTEGER</code> (<code>0</code> / <code>1</code>)	<code>true</code> / <code>false</code>	<code>true</code> , <code>false</code>
<code>int</code>	<code>int</code>	<code>bigint</code>	<code>INTEGER</code>	number	any int; <code>serial</code> only with <code>defaultPersist</code>
<code>double</code>	<code>double</code>	<code>double precision</code>	<code>REAL</code>	number	any double
<code>String</code>	<code>String</code>	<code>text</code>	<code>TEXT</code>	string	quoted <code>' ... '</code> or <code>" ... "</code>
<code>DateTime</code>	<code>DateTime</code>	<code>timestamp without time zone</code>	<code>INTEGER</code> (UTC epoch ms)	UTC ISO-8601 string	<code>now</code> , or <code>yyyy-MM- dd'T'HH: S'Z'</code>



YAML	Dart	Postgres	SQLite	JSON	Defaults
<code>Duration</code>	<code>Duration</code>	<code>bigint</code> (milliseconds)	<code>INTEGER</code> (ms)	int milliseconds	<code>Xd Xh Xmin Xs</code> <code>Xms</code>
<code>ByteData</code>	<code>ByteData</code>	<code>bytea</code>	<code>BLOB</code>	<code>decode('<base64>', 'base64')</code>	none
<code>UuidValue</code>	<code>UuidValue</code>	<code>uuid</code>	<code>BLOB</code> (16 bytes, no dashes)	UUID string	<code>random</code> , <code>random_v7</code> , or a quoted UUID
<code>Uri</code>	<code>Uri</code>	<code>text</code>	<code>TEXT</code>	<code>Uri.toString()</code>	quoted URI string
<code>BigInt</code>	<code>BigInt</code>	<code>text</code>	<code>TEXT</code>	decimal string	any <code>BigInt.parse</code> -able value

Vector types require a dimension of at least 1, for example `Vector(1536)`.

YAML	Dart	Postgres	SQLite	JSON	Defaults
<code>Vector(n)</code>	<code>Vector</code>	<code>vector(n)</code>	<code>TEXT</code>	<code>List<double></code>	none
<code>HalfVector(n)</code>	<code>HalfVector</code>	<code>halfvec(n)</code>	<code>TEXT</code>	<code>List<double></code>	none
<code>SparseVector(n)</code>	<code>SparseVector</code>	<code>sparsevec(n)</code>	<code>TEXT</code>	string <code>{i:v, ...}/n</code>	none
<code>Bit(n)</code>	<code>Bit</code>	<code>bit(n)</code>	<code>TEXT</code>	bit string	none

Geography types use SRID 4326 (WGS 84), which is not configurable per column.

YAML	Dart	Postgres	SQLite	JSON	Defaults
<code>GeographyPoint</code>	<code>GeographyPoint</code>	<code>geography(Point, 4326)</code>	<code>TEXT</code> (EWKT)	EWKT, for example <code>SRID=4326;POINT(lon lat)</code>	none
<code>GeographyLineString</code>	<code>GeographyLineString</code>	<code>geography(LineString, 4326)</code>	<code>TEXT</code>	EWKT	none
<code>GeographyPolygon</code>	<code>GeographyPolygon</code>	<code>geography(Polygon, 4326)</code>	<code>TEXT</code>	EWKT	none
<code>GeographyGeometryCollection</code>	<code>GeographyGeometryCollection</code>	<code>geography(GeometryCollection, 4326)</code>	<code>TEXT</code>	EWKT	none

Enums store as `text` or `bigint`, not json. A field default is any of the enum's values.

<code>serialized</code>	JSON	Postgres	SQLite
<code>byName</code> (default)	string literal	<code>text</code>	<code>TEXT</code>
<code>byIndex</code>	int index	<code>bigint</code>	<code>INTEGER</code>

Collections, records, nested models, custom classes, and `dynamic` persist as `json` or `jsonb`. See [JSON vs JSONB](#).

YAML	JSON	Postgres default	SQLite (<code>json</code>)	SQLite (<code>jsonb</code>)
<code>List<T></code>	JSON array	<code>json</code>	<code>TEXT</code>	<code>BLOB</code> via <code>jsonb()</code>
<code>Set<T></code>	JSON array (order not guaranteed)	<code>json</code>	<code>TEXT</code>	<code>BLOB</code> via <code>jsonb()</code>

YAML	JSON	Postgres default	SQLite (json)	SQLite (jsonb)
<code>Map<String, V></code>	JSON object	json	TEXT	BLOB via jsonb()
<code>Map<K, V></code> (K is not String)	JSON array of {k, v} objects	json	TEXT	BLOB via jsonb()
<code>(int, String) / ({int n})</code>	<code>{ "p": [...], "n": {...} }</code>	json	TEXT	BLOB via jsonb()
nested model / custom class	object JSON	json	TEXT	BLOB via jsonb()
dynamic	value plus type metadata	json	TEXT	BLOB via jsonb()

Nullability

A trailing `?` on the type makes the field nullable. Non-nullable fields are required constructor parameters. The `required` keyword is valid only on nullable fields: it makes the constructor parameter required while keeping the type nullable.

```
class: Person
fields:
  name: String
  nickname: String?, required
  age: int?
```

The `dynamic` type is already nullable. `dynamic?` and `List<dynamic?>` are rejected. See [Required fields](#) and [Dynamic fields](#).

Serialization

Three encodings are involved when a value moves between the app, the server, and the database:

1. **Protocol JSON** (client and server): `toJson()` / `fromJson()`.
 - `int`, `bool`, `double`, and `String` pass through as native JSON.
 - Every other type converts, as listed in the overview tables.

2. **Postgres literals** used when writing SQL: UUIDs are quoted, `ByteData` is `\x` hex, vectors are `'[1,2,3]'`, geography is EWKT, and collections are JSON text.

3. **SQLite literals** used by the [client-side database](#): `bool` is `0` / `1`, `DateTime` is epoch milliseconds, `UuidValue` is a 16-byte blob, `ByteData` is `X'hex'`, and `jsonb` values use `jsonb(...)`.

Protocol JSON for `DateTime` always uses `toUtc().toIso8601String()`. Postgres stores the value as `timestamp without time zone`, so stored values are UTC.

Primitive types

```
class: Sample
fields:
  active: bool
  count: int
  price: double
  name: String
  createdAt: DateTime
  timeout: Duration
  payload: ByteData
  uuid: UuidValue
  homepage: Uri
  huge: BigInt
```

bool

```
fields:
  published: bool, default=false
```

SQLite stores `false` as `0` and `true` as `1`, and reads those integers back as bools. Ordering operators (`>`, `<`, `≥`, `≤`) are not available. Equality operators are: `equals`, `notEquals`, `inSet`, `notInSet`.

int

```
fields:
  views: int, default=0
  invoiceNumber: int?, defaultPersist=serial
```

An `int` column is always 64-bit Postgres `bigint`, not `integer`. Serial IDs use `bigserial`. The `serial` default is valid only with `defaultPersist`, never with `default` or `defaultModel`. On SQLite, `serial` is supported only on the `id` column.

The `Duration` type also uses Postgres `bigint`. The column type is the same; the meaning is not. A `Duration` value is milliseconds.

double

```
fields:
  rating: double, default=0.0
```

On SQLite, `NaN` is stored as `NULL`, and `Infinity` is stored as `1e999`.

String

```
fields:
  title: String, default='Untitled'
```

Strings are unbounded `text` (Postgres) or `TEXT` (SQLite). There is no `varchar` length. String columns support `like`, `notLike`, `ilike`, and `notILike` in addition to ordering and equality. On SQLite, `like` and `ilike` are both case-insensitive for ASCII characters. See [Filter](#).

DateTime

```
fields:
  createdAt: DateTime, default=now
  publishedAt: DateTime?, default=2024-05-01T22:00:00.000Z
```

Values are always converted to UTC. The Postgres default for `now` is `CURRENT_TIMESTAMP`. SQLite stores UTC epoch milliseconds as `INTEGER`.

On an `immutable class`, `now` is not valid with `default` or `defaultModel`. A table model can set `defaultPersist=now` instead.

Duration

```
fields:
  timeout: Duration, default=1d 2h 10min 30s 100ms
```

Stored as milliseconds in a `bigint` (Postgres) or `INTEGER` (SQLite) column, the same SQL type as `int`. JSON is that millisecond count as an integer.

ByteData

```
fields:
  blob: ByteData
```

No YAML default. Postgres inserts use `\x` hex for `bytea`. SQLite uses a `BLOB` with `X'hex'` literals. On the wire, the protocol JSON is a Postgres-style string: `decode('<base64>', 'base64')`.

UuidValue

```
fields:
  publicId: UuidValue, default=random
  orderedId: UuidValue, default=random_v7
  fixedId: UuidValue, default='550e8400-e29b-41d4-a716-446655440000'
```

The `random` default generates a UUID v4 (`Uuid().v4obj()` in Dart, `gen_random_uuid()` in Postgres). The `random_v7` default generates a UUID v7 (`Uuid().v7obj()` in Dart, `gen_random_uuid_v7()` in Postgres). SQLite stores the 16-byte value as a `BLOB` with no dashes.

On an `immutable class`, `random` and `random_v7` are not valid with `default` or `defaultModel`. A table model can set `defaultPersist` instead.

Uri

```
fields:
  homepage: Uri, default='https://serverpod.dev'
```

A URI default must be a quoted string. Stored as `text` / `TEXT`. Query operators are equality only (`equals`, `notEquals`), not ordered comparison.

BigInt

```
fields:
  huge: BigInt, default='1234567890'
```

Stored as `text` / `TEXT`, not a numeric column. The JSON value is a decimal string. Any value `BigInt.parse` accepts is a valid default. Query operators are equality only (`equals`, `notEquals`).

Enums

```
enum: Animal
serialized: byName
default: unknown
values:
  - unknown
  - dog
  - cat
```

The `serialized` keyword has two values. `byName` is the default and stores the string literal as Postgres `text` / SQLite `TEXT`. `byIndex` stores the index as Postgres `bigint` / SQLite `INTEGER`.

Changing the order of a `byIndex` enum changes the stored integers and can corrupt existing data. The default `byName` mode is stable when values are added or reordered.

A field default is any of the enum's values. The enum's own `default` is the fallback when an unknown value is deserialized, not a column default. See [Handling unknown enum values](#).

Enhanced enum [properties](#) support `int`, `double`, `bool`, `String`, and their nullable forms. Those properties exist only on the generated Dart enum; they are not extra columns.

Collections and records

Type arguments are required: `List<String>`, `Map<String, int>`, `Set<UuidValue>`. Untyped `List` or `Map` is rejected.

```
class: Packed
fields:
  tags: List<String>
  uniqueTags: Set<String>
  counts: Map<String, int>
  labeled: Map<int, String>
  pair: (int, String)
  named: ({int count})
```

- `List<T>` is a JSON array.
- `Set<T>` is also a JSON array. Order is not guaranteed after a round trip.
- `Map<String, V>` is a JSON object.
- `Map<K, V>` where `K` is not `String` is a JSON array of `{k, v}` objects, not a JSON object, because JSON object keys must be strings.
- Records serialize positional fields under `"p"` and named fields under `"n"`. A record with only positional fields omits `"n"`; a record with only named fields omits `"p"`. A single-element record needs a trailing comma: `(int,)`.

```
{
  "pair": { "p": [1, "hello"] },
  "named": { "n": { "count": 42 } },
  "labeled": [{ "k": 1, "v": "a" }, { "k": 2, "v": "b" }]
}
```

No YAML defaults for `List`, `Map`, `Set`, or records.

Nested models

A nested model (or custom class) in a JSON column is a copy stored on that row. Updating it does not update other rows that hold a similar object. A `relation` field stores a foreign key instead, so multiple rows can point at the same object.

```

class: Company
table: company
fields:
  address: Address
  billingAddress: Address?, relation

```

The `address` field is stored as json on `company`. The `billingAddress` field is a foreign key to an `address` row. See [Tables](#).

JSON vs JSONB

Serializable types (`List`, `Map`, `Set`, records, nested models, custom classes, and `dynamic`) default to Postgres `json` (SQLite `TEXT`). Setting `serializationDataType=jsonb` on the field, the class, or `serialize_as_jsonb_by_default: true` in `config/generator.yaml` stores them as:

- Postgres: `jsonb`, which can use a [GIN index](#)
- SQLite: `BLOB` via SQLite's `jsonb()` function

The `serializationDataType` keyword is not valid on primitives (`String`, `int`, `bool`, and the other core types). A GIN index requires every indexed field to be jsonb. See [Storing serializable fields as JSONB](#).

Vector types

```

class: Document
table: document
fields:
  embedding: Vector(1536)

```

Dimension is required: `Vector(1536)`, `HalfVector(1536)`, `SparseVector(10000)`, `Bit(256)`. The dimension must be at least 1.

SQLite has no native vector type. Values are stored as text, vector query operators are not supported, and HNSW / IVFFLAT indexes are omitted.

On Postgres, the default index type is `hnsw`. `Vector`, `HalfVector`, and `Bit` also allow `ivfflat`. `SparseVector` allows only `hnsw`. Distance functions are `l2`, `innerProduct`, `cosine`, and `l1`. `Bit` also has Hamming and Jaccard.

Vector fields require the pgvector extension. See [Vector and geography fields](#) for usage, Dart APIs, and setup.

Geography types

```
class: Store
table: store
fields:
  location: GeographyPoint
```

Coordinates in EWKT are longitude then latitude: `SRID=4326;POINT(lon lat)`. No YAML defaults.

Indexes are `gist` (default) or `spgist` only. Query operators are `intersects`, `distanceWithin`, `distance`, `contains`, and `within` (Postgres `ST_*` / `ST_GeogFromText`).

Geography fields require PostGIS. On SQLite they are opaque EWKT strings: CRUD round-trips, spatial operators throw. See [Vector and geography fields](#) and [Upgrading to PostGIS support](#).

Dynamic and custom classes

A `dynamic` field holds any serializable value when the type is not known at compile time. Serverpod includes type metadata in the JSON so the value round-trips. That wrapper is an implementation detail. See [Dynamic fields](#).

A hand-written Dart class can be a field type after it is registered in `config/generator.yaml`. It persists as json / jsonb like a nested model. See [Custom serialization](#).

ID types

If `id` is omitted on a table model, it is `int?` with `defaultPersist=serial`.

Type	YAML	Persist default	Model default	Postgres	SQLite
<code>int</code>	<code>id: int?, defaultPersist=serial</code>	<code>serial</code>	not allowed	<code>bigserial PRIMARY KEY</code>	<code>INTEGER PRIMARY KEY (ROWID)</code>
<code>UuidValue</code>	<code>id: UuidValue?, defaultPersist=random</code>	<code>random, random_v7</code>	<code>random, random_v7</code>	<code>uuid</code>	<code>BLOB</code>

An `int` id must be nullable. `serial` is `defaultPersist` only, never `defaultModel`. SQLite allows autoincrement only on the `id` column.

Choosing between `int` and `UuidValue`, and generating the id before insert with `defaultModel=random`, is covered in [Choosing an ID strategy](#).

Defaults

Allowed values by type are in the [overview](#) and in each type section. The `default`, `defaultModel`, and `defaultPersist` keywords are explained in [Default values](#).

These types cannot have a YAML default:

- `ByteData`
- Vector types (`Vector`, `HalfVector`, `SparseVector`, `Bit`)
- Geography types
- `List`, `Map`, `Set`
- Records
- Nested models and custom classes
- `dynamic`

On an immutable class, `now`, `random`, and `random_v7` are not valid with `default` or `defaultModel`. Those values belong on `defaultPersist` instead.

Indexes

Fields marked `!persist` cannot be indexed.

Fields	Default index	Allowed
Scalars	<code>btree</code>	<code>btree</code> , <code>hash</code> , <code>gin</code> , <code>gist</code> , <code>spgist</code> , <code>brin</code>
jsonb serializable	<code>gin</code>	<code>gin</code> requires every indexed field to be jsonb
<code>Vector</code> / <code>HalfVector</code> / <code>Bit</code>	<code>hnsu</code>	<code>hnsu</code> , <code>ivfflat</code>
<code>SparseVector</code>	<code>hnsu</code>	<code>hnsu</code> only
Geography	<code>gist</code>	<code>gist</code> , <code>spgist</code>

Index types other than `btree` are Postgres-only. On SQLite they are skipped when a migration is created. See [Indexing](#).

Query operators

Table columns expose the operators below in [filters](#).

Types	Operators
int, double, String, DateTime, Duration, UuidValue, enums	equals, notEquals, >, <, ≥, ≤, inSet, notInSet
int, double, DateTime, Duration	between, notBetween
String	like, notLike, ilike, notILike
bool	equals, notEquals, inSet, notInSet
Uri, BigInt	equals, notEquals
Vector, HalfVector, SparseVector	distanceL2, distanceInnerProduct, distanceCosine, distanceL1
Bit	distanceHamming, distanceJaccard
Geography	intersects, distanceWithin, distance, contains, within

Vector and geography operators run on Postgres. They are not supported on SQLite.

Postgres vs SQLite

The [client-side database](#) is SQLite. These mappings differ from Postgres:

Concern	Postgres	SQLite
bool	boolean	INTEGER 0 / 1
int	bigint	INTEGER
DateTime	timestamp without time zone	UTC epoch milliseconds
UuidValue	uuid	16-byte BLOB
ByteData	bytea (\x hex)	BLOB (X'hex')

Concern	Postgres	SQLite
<code>json</code> collections	<code>json</code>	<code>TEXT</code>
<code>jsonb</code> collections	<code>jsonb</code>	<code>BLOB</code> via <code>jsonb()</code>
Vectors	pgvector types, HNSW / IVFFLAT	<code>TEXT</code> , no vector indexes or operators
Geography	PostGIS geography, spatial operators	EWKT <code>TEXT</code> , no spatial operators
<code>serial</code>	any <code>int</code> column with <code>defaultPersist=serial</code>	<code>id</code> column only
<code>like</code> / <code>ilike</code>	<code>like</code> is case-sensitive	both are case-insensitive for ASCII
Non- <code>btree</code> indexes	created as declared	omitted, with a warning

The `Duration` and `int` types share a 64-bit integer column on both dialects. The `BigInt` type is text on both, not a numeric type.

Related

- [Working with models](#): model file format, required fields, and default keywords.
- [Tables](#): table models, ID types, and json vs jsonb.
- [Vector and geography fields](#): pgvector and PostGIS usage.
- [Dynamic fields](#): values whose type varies at runtime.
- [Custom serialization](#): hand-written Dart classes as field types.
- [Indexing](#): index types, GIN, HNSW, and geography indexes.
- [Filter](#): how to write the operators above.
- [Client-side database](#): SQLite on the device.
- [Model reference](#): every keyword in a model file.